

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA14

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 25

Duration :60 Mins
On Class
Total Marks:50

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I V G ABBIRAMY - 192524097 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

V G ABBIRAMY

Annexure A

SHORT ANSWER TEST

Q1. A lexical analyzer produces the following token stream:

id + id

Explain how the parser validates the syntactic structure of the expression.

Q2. Given the grammar:

$S \rightarrow aS \mid b$

Determine whether the string aaab belongs to the language using leftmost derivation.

Q3. Remove left recursion from the grammar:

$A \rightarrow Aa \mid b$

Q4. Find FIRST(S) for the grammar:

$S \rightarrow aA \mid b$

$A \rightarrow c \mid \epsilon$

Q5. Determine whether the grammar is suitable for top-down parsing.

$S \rightarrow Sa \mid b$

Justify your answer.

Q6. Apply left factoring to the grammar:

$S \rightarrow \text{if } E \text{ then } S$

$S \rightarrow \text{if } E \text{ then } S \text{ else } S$

Q7. Construct a parse tree for the string:

a + b

using the grammar:

$E \rightarrow E + E$

$E \rightarrow \text{id}$

Q8. Write the recursive descent procedures for the grammar:

$S \rightarrow aA$

$A \rightarrow b$

Q9. Given the grammar:

$S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b$

Construct the predictive parsing table.

Q10. Demonstrate the first three actions of a shift-reduce parser for the input:

id + id

Q11. Identify the handle in the string:

id + id

for the grammar:

$E \rightarrow E + E$

$E \rightarrow \text{id}$

Q12. Construct the operator precedence relations for the operators:

+
*

where * has higher precedence than +.

Q13. Explain how operator precedence parsing handles the expression:
 $\text{id} * \text{id}$

Q14. What is an LR parser? Illustrate the parsing process using the grammar:
 $S \rightarrow aA$
 $A \rightarrow b$

Q15. Given the grammar:
 $S \rightarrow aA$
 $A \rightarrow b$
Write the steps involved in constructing an SLR parsing table.

Q16. Compute FOLLOW(A) for the grammar:
 $S \rightarrow AB$
 $A \rightarrow a$
 $B \rightarrow b$

Q17. Construct the augmented grammar for:
 $S \rightarrow a$

Q18. Find the closure of the LR(0) item:
 $S' \rightarrow \bullet S$
for the grammar:
 $S \rightarrow a$

Q19. Check whether the grammar is ambiguous:
 $E \rightarrow E + E$
 $E \rightarrow \text{id}$
using the string:
 $\text{id} + \text{id} + \text{id}$

Q20. Construct a leftmost derivation for the string:
 $\text{id} + \text{id}$
using the grammar:
 $E \rightarrow E + E$
 $E \rightarrow \text{id}$

Q21. Apply left factoring to the grammar:
 $S \rightarrow aA$
 $S \rightarrow aB$
 $A \rightarrow b$
 $B \rightarrow c$

Q22. Find FIRST(A) for the grammar:
 $A \rightarrow x$
 $A \rightarrow y$

Q23. Determine whether the grammar is suitable for predictive parsing.
 $S \rightarrow aA \mid aB$
 $A \rightarrow b$
 $B \rightarrow c$

Justify your answer.

Q24. Construct recursive descent parser procedures for the grammar:

$S \rightarrow aS$

$S \rightarrow b$

Q25. Build the predictive parsing table for the grammar:

$S \rightarrow aS$

$S \rightarrow b$

Show the table entries for the terminals:

a, b, \$

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

SHORT ANSWER TEST

NAME : V. G. ABBIRAM

REG No : 192524077

COURSE CODE : CSA1408

COURSE NAME : Compiler
Design

1) $id + id$

$$E \rightarrow E + E / id$$

* The parser checks whether the token sequence $id + id$ follows the grammar rules & validate its syntax.

2) Grammar :

$$S \rightarrow aS / b$$

$$\Rightarrow S \rightarrow aS$$

$$S \rightarrow aaS$$

$$\Rightarrow aaaS$$

$$\Rightarrow aaab$$

3) Given :

$$A \rightarrow Aa / b$$

After removing :

$$A \rightarrow bA'$$

$$A' \rightarrow aA' / \epsilon$$

4) Grammar :

$$S \rightarrow aA / b$$

$$A \rightarrow c / \epsilon$$

$$\text{First}(S) = \{a, b\}$$

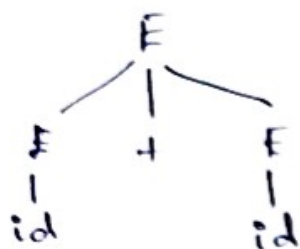
5) Yes the grammar is suitable for top-down parsing because the production starts with different terminals.

6) Left factored grammar :

$$* S \rightarrow \text{if } F \text{ then } SS'$$

$$* S' \rightarrow \text{else } S / \epsilon$$

7) Parse tree for $a + b$:



8) Recursive-descent Procedure:

$S()$:

$\text{match}(a)$

$A()$

$A()$:

$\text{match}(b)$

9) Predictive Parsing Table:

	a	b
S	$S \rightarrow AB$	-
A	$A \rightarrow a$	-
B	-	$B \rightarrow b$

10) First three actions of Shift-reduce parser:

1. Shift id 2. Reduce $id \rightarrow E$ 3. Shift $+$

11) Handle: id

12) $+$ < $*$

$*$ > $+$

13) The parser shifts $*$ before reducing because $*$ has higher precedence than $+$.

14) An LR parser is a bottom-up parser that scans input from left to right and constructs the rightmost derivation in reverse.

- 15)
1. Augment the grammar
 2. Construct LR(0) items
 3. Build DFA
 4. Compute follow sets
 5. Fill action and GOTO tables.

16) $\text{FOLLOW}(A) = \{b\}$

17) Augmented grammar: $S' \rightarrow S$
 $S \rightarrow a$

18) Closure: $S' \rightarrow \cdot S$
 $S \rightarrow \cdot a$

19) Ambiguous:

The string $id + id + id$ has more than one valid parse tree.

20) $E \rightarrow E + E$

$\Rightarrow id + E$

$\Rightarrow id + id$

21) Left Factoring:

$S \rightarrow ax$

$x \rightarrow A|B$

$A \rightarrow b$

$B \rightarrow c$

22) $\text{first}(A) = \{x, y\}$

23) No. Both productions start with a , causing a FIRST/FIRST conflict.

24) $S()$

```
{  
  if (input == 'a')  
  {  
    match(a);  
    S();  
  }  
  else  
    match(b);  
}
```

	a	b	\$
S	$S \rightarrow aS$	$S \rightarrow b$	—